

Experiment ‘p_13_8_recursive_largest_seq’ Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_13_8_recursive_largest_seq

Natural language statement: Write a recursive method that takes an array and returns the largest element in the array. Hint: Find the largest element in the subset containing all but the last element. Then compare that maximum to the value of the last element.

Method signature: p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)

Ensures

- `largest == find_largest(inputs)`

Requires

- `|inputs| >= 1`

Functional Code Given

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
  }
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a recursive method that takes an array and returns the largest element in the array
→ . Hint: Find the largest element in the subset containing all but the last element
→ . Then compare that maximum to the value of the last element.

The signature should be:

`p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)`

The method should respect the following contract:

`requires |inputs| >= 1, ensures largest == find_largest(inputs)`

The contract uses the following dafny code:

```
function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnAlnTW6xy5dGyZhjwDOu6AE6IQIo

Model: gpt-5.5-2026-04-23

Created at: 1780611843

Finish reason: stop

Usage: 596 tokens in, and 618 tokens out

```
{
  if |inputs| == 1 {
    largest := inputs[0];
  } else {
```

```

    var sub_largest := p_13_8_recursive_largest_seq(inputs[1..]);
    if inputs[0] >= sub_largest {
        largest := inputs[0];
    } else {
        largest := sub_largest;
    }
}
}

```

Final Program

```

function find_largest(inputs: seq<int>): int
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> find_largest(inputs) >= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert inputs == [inputs[0]] + inputs[1..];
    var sub_largest := find_largest(inputs[1..]) ;
    if inputs[0] >= sub_largest then inputs[0] else sub_largest
  }

method p_13_8_recursive_largest_seq(inputs: seq<int>) returns (largest: int)
  requires |inputs| >= 1
  ensures largest == find_largest(inputs)
{
  if |inputs| == 1 {
    largest := inputs[0];
  } else {
    var sub_largest := p_13_8_recursive_largest_seq(inputs[1..]);
    if inputs[0] >= sub_largest {
      largest := inputs[0];
    } else {
      largest := sub_largest;
    }
  }
}
}

```

Total Token Usage

Input tokens: 596

Output tokens: 618

Reasoning tokens: 512

Sum of ‘total tokens’: 1214

Experiment Timings

Overall Experiment started at 1780611842662, ended at 1780611871117, lasting 28455ms (28.46 seconds)

Iteration #1 started at 1780611842669, ended at 1780611871116, lasting 28447ms (28.45 seconds)